BTTF CPP - TRACK NETWORK

Context_

You previously worked on the R-Type project's network track. This BTTF focuses on core networked game mechanics: client-side prediction, server authority, and state synchronization.

Objective

Build a simple multiplayer top-down shooter demonstrating fundamental networked game architecture.

01 Requirements

Core Mechanics

- Player movement (WASD or arrow keys)
- Mouse aiming (player sprite/direction follows mouse position)
- Shooting (mouse click spawns projectiles)
- Simple radius-based collision with other players

Network Architecture

- UDP protocol for client-server communication
- Client-side prediction: local player moves immediately on input
- Server authority: server validates all actions and simulates game state
- State broadcasting: server sends world state to all clients
- Remote player interpolation: smooth movement between state updates (no teleporting/snapping)

Think

- What is interpolation?
- How can the client game interpolate between previous data and new data
- Where is interpolation possible and where is it not

Also make sure to understand what client-side prediction is:
https://www.youtube.com/watch?v=L6Nih4T0iI4

Technical Requirements

Client

- Capture local input (movement, mouse position, shooting)
- Predict local player position immediately
- Send input commands to server
- Receive authoritative state from server
- Reconcile local prediction with server state
- Interpolate remote player positions for smooth rendering

Technical Requirements

Server

- Receive input from all clients
- Validate input (speed limits, bounds checking)
- Simulate authoritative game state
- Handle shooting events (spawn projectiles, broadcast to clients)
- Broadcast game state to all connected clients
- Detect and handle client disconnections

Scope Limitations_

Focus on network architecture, not game features:

- No health system required
- No enemy AI required
- No scoring system required
- No complex game logic beyond movement and shooting

The goal is demonstrating proper client/server split and state synchronization.

02 Validation

Validation_

Your implementation will be evaluated during live demonstration:

- Run server on teacher's machine
- Run 2+ clients
- Teacher will observe smoothness, responsiveness, and correct behavior
- Teacher may inject artificial latency to test interpolation
- Teacher has final say on what constitutes "smooth"

Thanks